

¿Qué es paradigma orientado a objetos?

Es un enfoque de programación basado en definir e interactuar unidades mínimas de código llamadas objetos, que agrupan datos (atributos) y comportamiento (métodos), para modelar y resolver problemas del mundo real.

¿Qué propiedades tiene el paradigma orientado a objetos?

Encapsulamiento (agrupar datos y métodos, ocultando detalles internos), abstracción (modelar solo lo relevante de un objeto real), herencia (reutilizar código creando clases a partir de otras) y polimorfismo (un mismo método se comporta distinto según el objeto que lo ejecuta).

¿Qué es una clase?

Es una plantilla o molde que define los atributos y las operaciones (métodos) que tendrán los objetos creados a partir de ella.

¿Cómo se crea una clase en Python?

Con la palabra reservada `class`, seguida del nombre de la clase (por convención empieza en mayúscula) y dos puntos. Ejemplo: `class Estudiante:` `pass`

¿Qué es un objeto?

Es una instancia concreta de una clase: un componente con un rol específico, que tiene atributos (características) y comportamiento (operaciones que puede realizar), y que puede interactuar con otros objetos.

¿Qué elementos componen un objeto?

Un conjunto de atributos (características o datos) y un conjunto de métodos (comportamiento u operaciones que puede ejecutar).

¿Qué es un objeto de clase?

Es la propia clase vista como objeto. En Python las clases también son objetos, así que se les puede hacer referencia a sus atributos y métodos sin necesidad de instanciarlas primero.

¿Cómo se crea un objeto de clase?

Se crea automáticamente cuando el intérprete ejecuta la sentencia `class NombreClase::`; en ese momento Python construye el objeto clase con su propio espacio de nombres.

¿Qué es un objeto de instancia?

Es el objeto que resulta de instanciar una clase: un ejemplar concreto creado a partir del molde definido por esa clase, con su propio espacio de nombres.

¿Cómo se instancia un objeto?

Llamando a la clase como si fuera una función, pasando los argumentos que pida su constructor: `variable = NombreClase(argumentos)`.

¿Qué son los atributos?

También llamados variables de instancia; son las características o datos que describen el estado de un objeto. Se acceden con `self.atributo` dentro de la clase.

¿Qué son los métodos?

Son funciones definidas dentro de una clase que representan el comportamiento de un objeto. Reciben como primer parámetro una referencia al objeto (`self`), salvo los métodos estáticos.

¿Qué es un método de clase?

Un método decorado con `@classmethod` que recibe como primer parámetro la referencia a la clase (`cls`) en lugar de a la instancia, y puede invocarse directamente sobre la clase sin crear un objeto.

¿Qué es un método de objeto?

Es un método de instancia: actúa sobre un objeto en particular, recibe self como primer parámetro y puede acceder o modificar sus atributos.

¿Qué son variables de clase?

Son variables compartidas por todos los objetos de una clase. Se declaran dentro de la clase pero fuera de los métodos, y su valor es el mismo para todas las instancias mientras no se sobrescriba a nivel de instancia.

¿Qué son variables de instancia?

Son los atributos propios de cada objeto, declarados con self dentro del constructor u otros métodos. Cada instancia tiene su propia copia independiente.

¿Qué son métodos static?

Métodos decorados con @staticmethod que no reciben ninguna referencia implícita (ni self ni cls), por lo que no pueden acceder a atributos de instancia ni de clase; funcionan como funciones normales agrupadas dentro de la clase por relación lógica.

¿Qué es un constructor de clase?

Es el mecanismo que se ejecuta al crear una instancia para inicializarla. En la práctica se implementa con __init__(), aunque el método realmente encargado de crear el objeto es __new__().

¿Qué es un destructor de clase?

Es el método especial __del__(), que Python invoca automáticamente cuando el contador de referencias de un objeto llega a cero y el recolector de basura lo elimina de memoria.

¿Qué es la visibilidad de miembros de clase?

Se refiere a qué tan accesible es un atributo o método desde fuera de la clase. Python no tiene modificadores public/private reales; usa una convención: un guion bajo (_atributo) indica uso protegido/interno, y doble guion bajo (__atributo) activa el "name mangling", dificultando el acceso directo desde fuera.

¿Qué son métodos static?

(Misma respuesta que la pregunta 17, está repetida en el cuestionario.)

¿Qué hace un método __init__()?

Se ejecuta automáticamente justo después de crear la instancia (después de __new__) y se usa para inicializar sus atributos con los valores recibidos como parámetros.

¿Qué función cumple el método __new__()?

Es el método realmente responsable de crear la instancia. Se ejecuta antes que __init__, recibe cls como primer parámetro, y solo si devuelve una instancia de la clase se llamará después a __init__.

Referencias bibliográficas:

Python Software Foundation. (s.f.). 9. *Classes* — *The Python Tutorial*.

<https://docs.python.org/3/tutorial/classes.html>

GeeksforGeeks. (2024). *Python Classes and Objects*.

<https://www.geeksforgeeks.org/python/python-classes-and-objects/>

Real Python. (s.f.). *Instance, Class, and Static Methods Demystified*.

<https://realpython.com/instance-class-and-static-methods-demystified/>

GeeksforGeeks. (2025). *Python staticmethod() Function*.

<https://www.geeksforgeeks.org/python-staticmethod-function>

Dev Community (pila). (2019). *Constructors in Python (**init** vs **new**)*.

<https://dev.to/pila/constructors-in-python-init-vs-new-2f9j>

GeeksforGeeks. (2025). *Private Variables in Python*.

<https://www.geeksforgeeks.org/python/private-variables-python/>

UniversoPython. (2025). *Python Encapsulation: Private Attributes and Properties*.

<https://universopython.com/en/blog/python-encapsulation-complete-guide>